Cloud Threat Detection System

Objective

To design and deploy a cloud-based system that automatically simulates and logs suspicious activities (e.g., failed logins) using AWS services, storing the data in a database for monitoring and analysis.

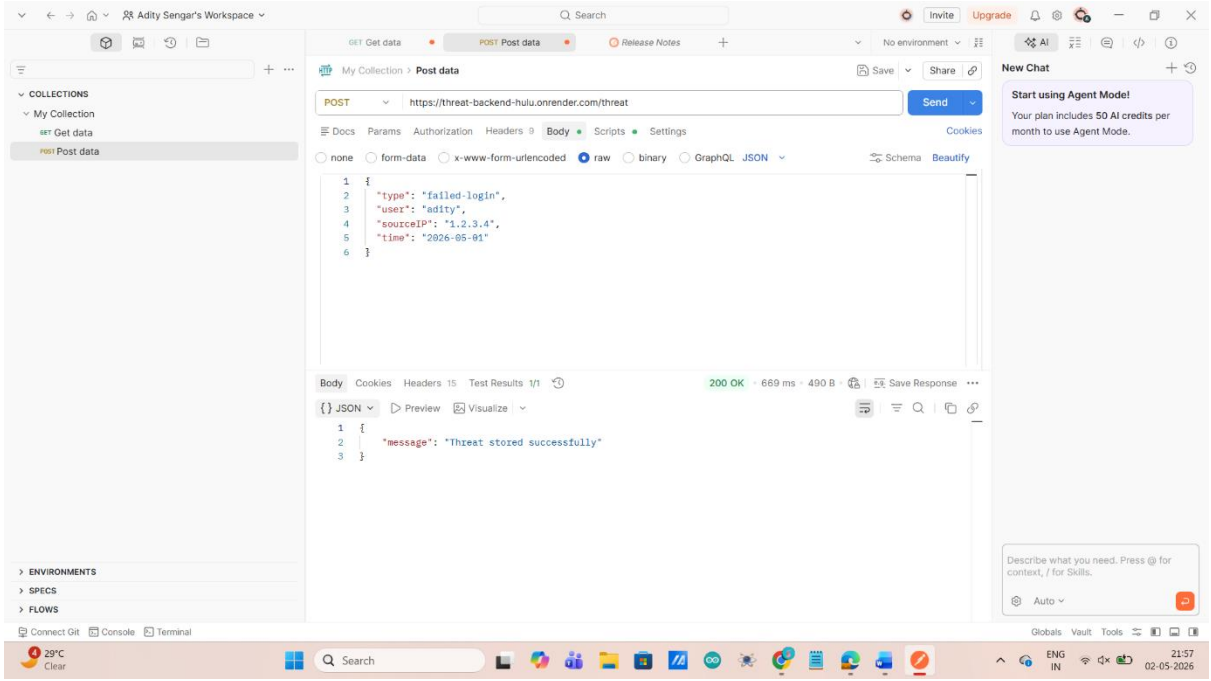
Tech Stack

- **AWS Lambda –** Serverless function to generate/send threat data
- **AWS EventBridge Scheduler –** Triggers Lambda periodically
- **Node.js (Express) –** Backend API
- **MongoDB Atlas –** Database for storing threats
- **Render –** Backend deployment
- **GitHub –** Version control

Implementation Steps

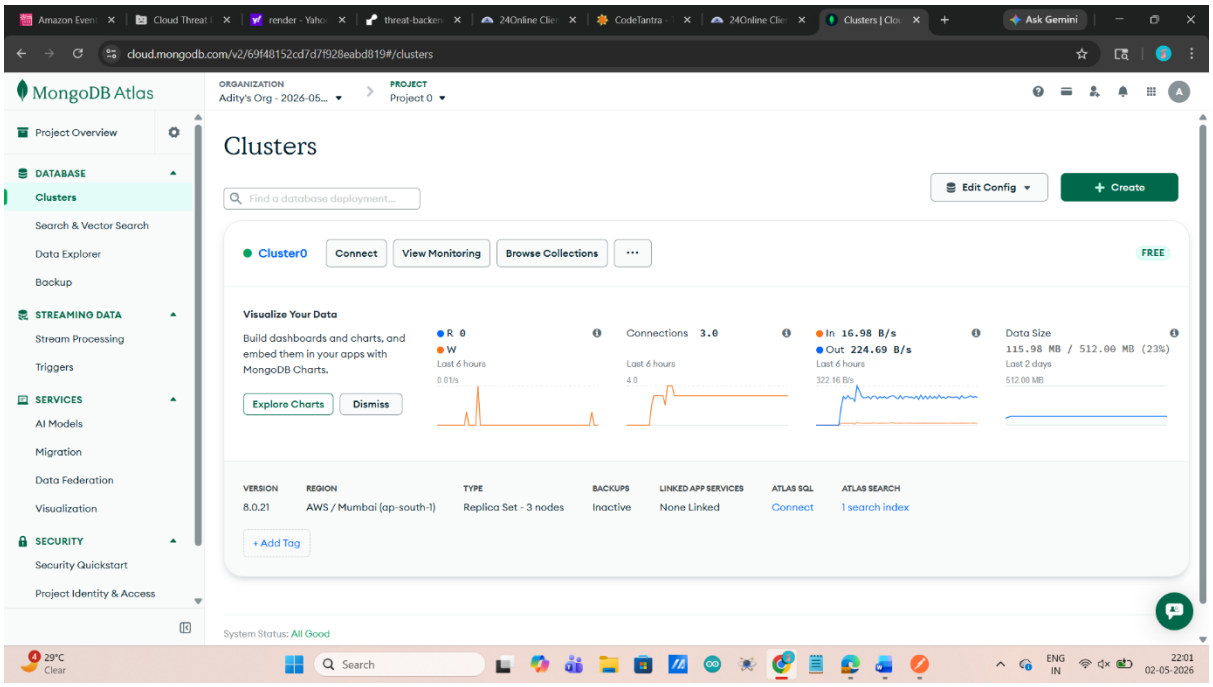
1. Backend Development (Node.js)

- **Created REST APIs:**
    - **POST /threat -** stores threat data
    - **GET /threats -** retrieves stored threats



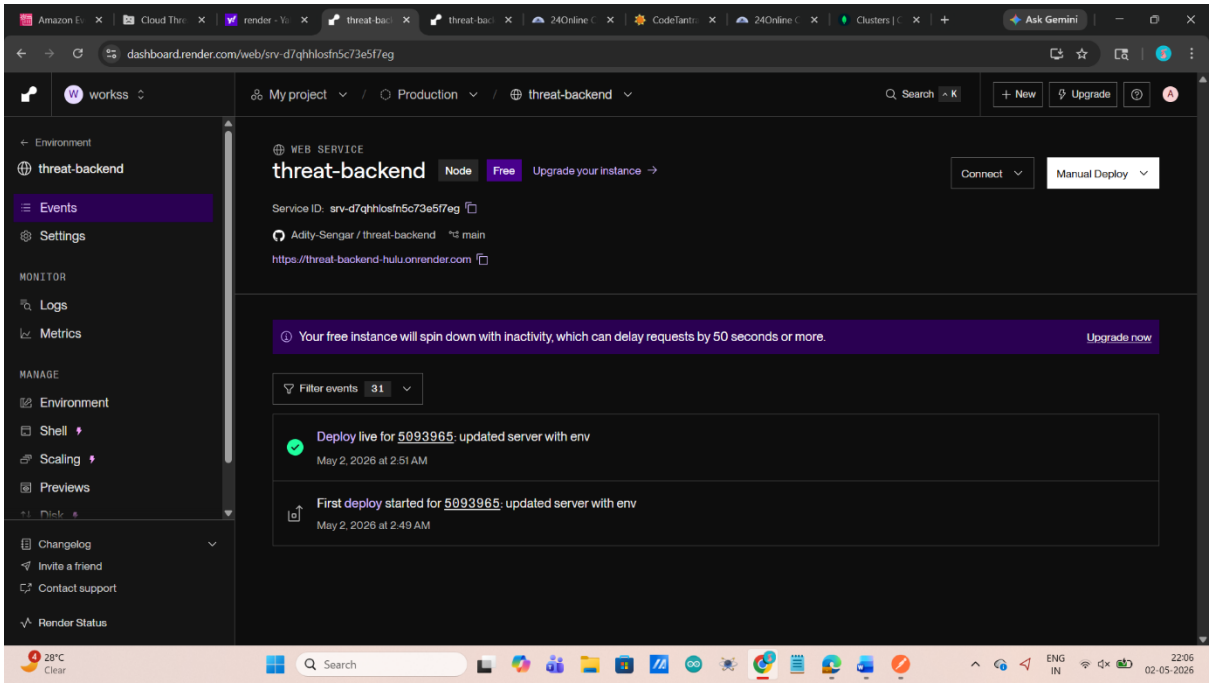
2. Database Setup (MongoDB Atlas)

- Created cluster and database threatDB
- Configured IP access and database user
- Verified data storage via Atlas dashboard



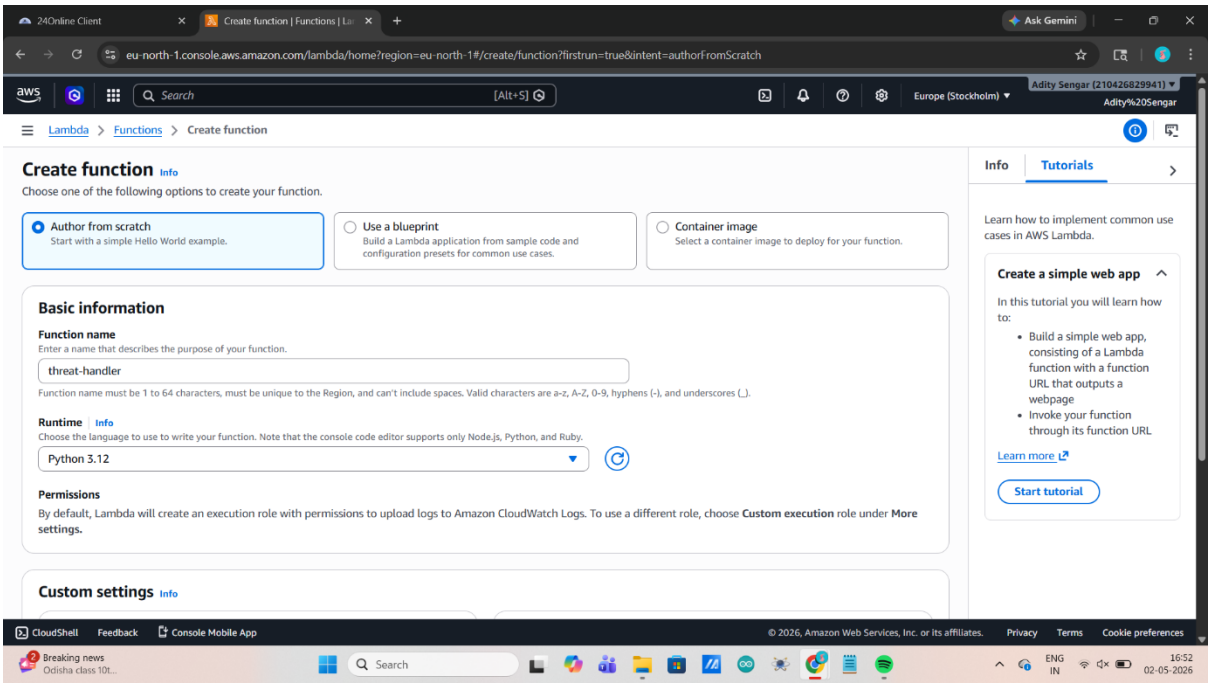
3. Deployment (Render)

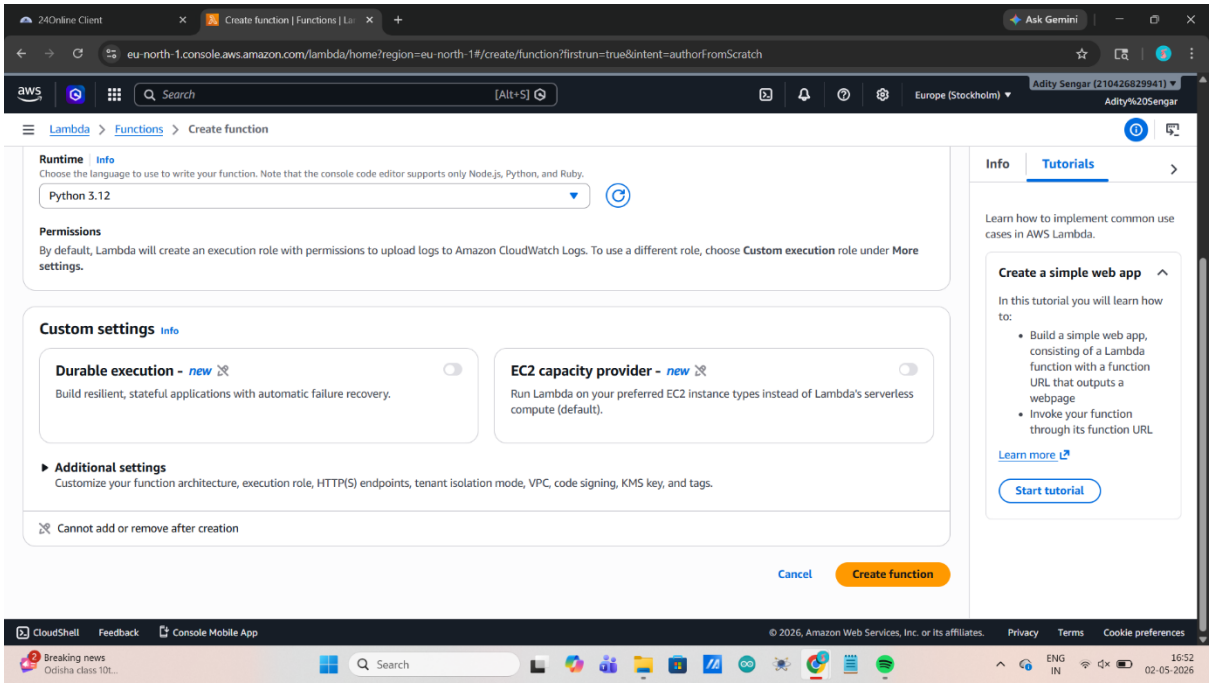
- Deployed Node.js backend as a web service
- Configured environment variables (.env)
- Verified live API endpoint

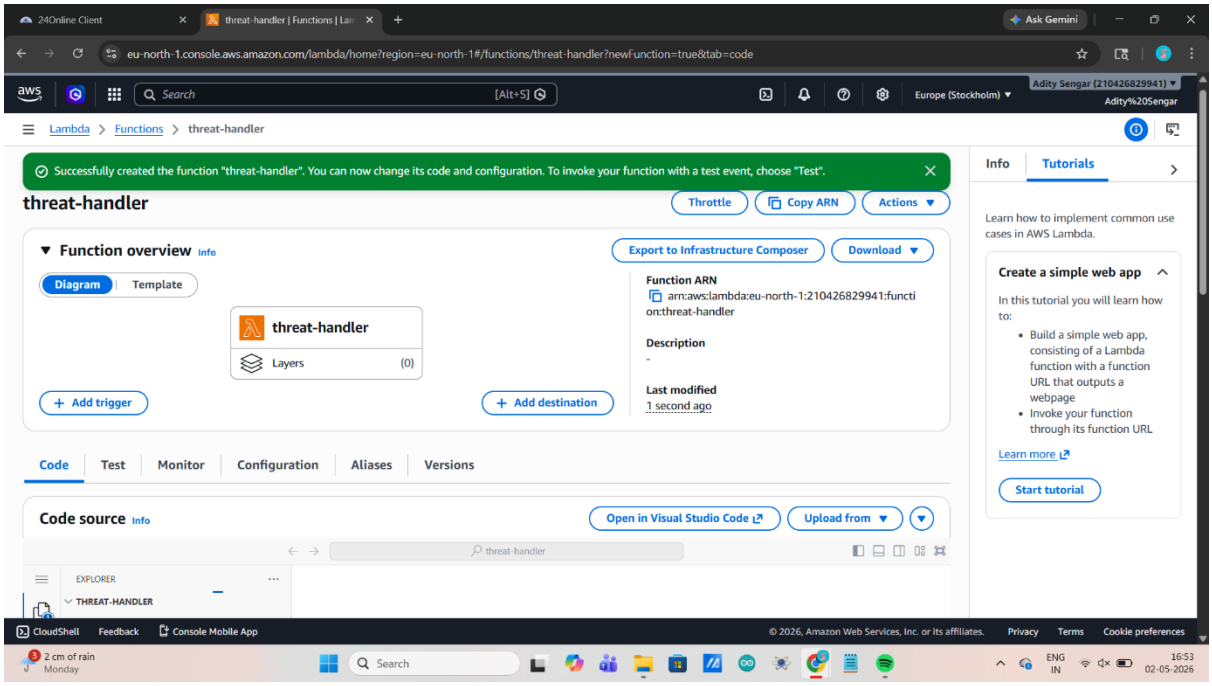


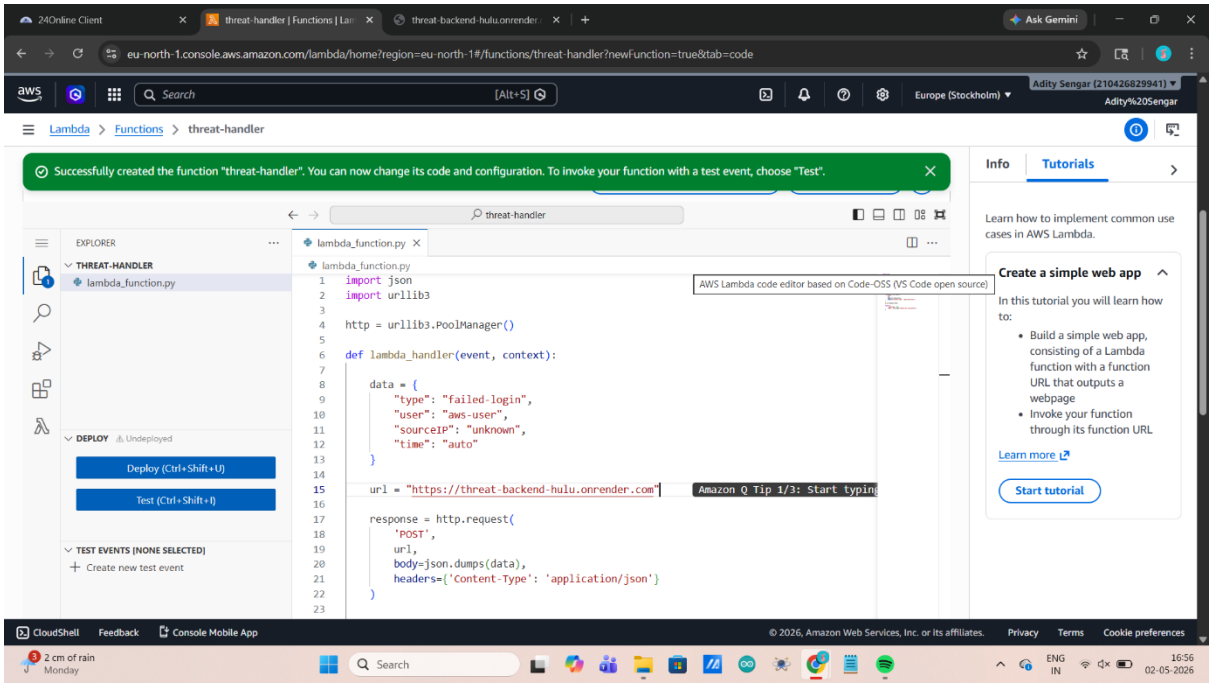
4. AWS Lambda Function

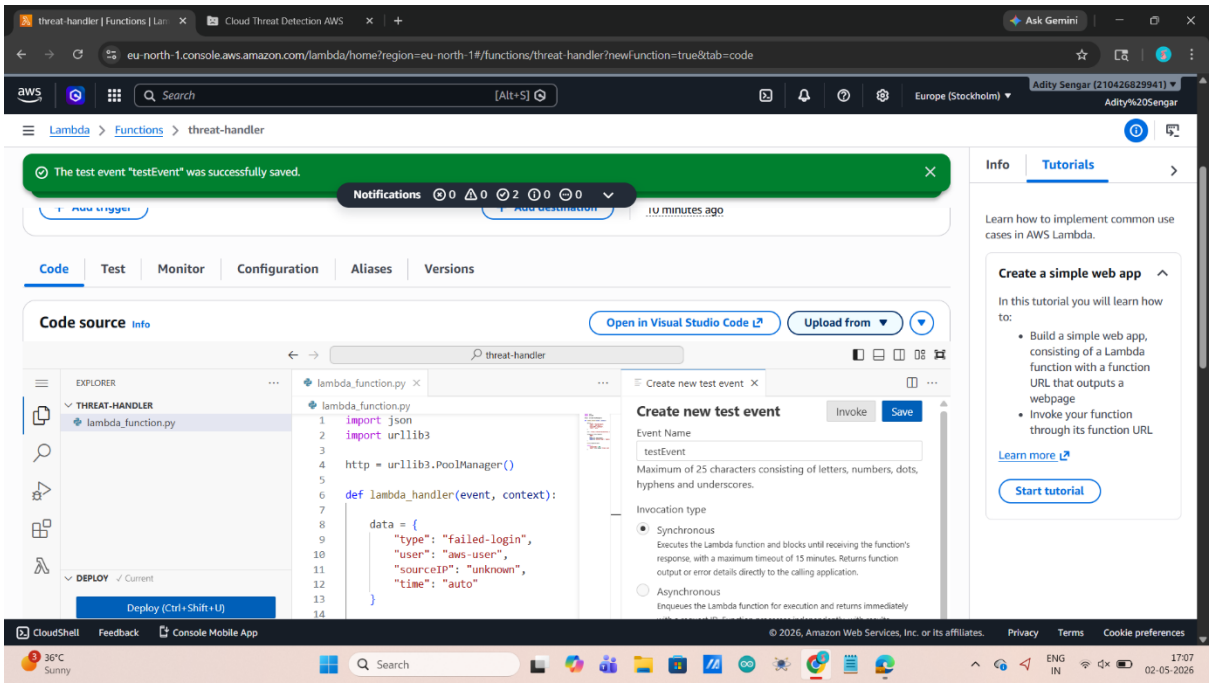
- Created a Lambda function in Python
- Function sends simulated threat data to backend API
- Used HTTP request (requests library)




24Online Client
Create function | Functions | La
Ask Gemini
eu-north-1.console.aws.amazon.com/lambda/home?region=eu-north-1#/create/function?firstrun=true&intent=authorFromScratch
Search
[Alt+S]
Europe (Stockholm)
Adity Sengar (210426829941)
Adity%20Sengar
Lambda > Functions > Create function
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.12
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under More settings.
Custom settings Info
Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Additional settings
Customize your function architecture, execution role, HTTP(S) endpoints, tenant isolation mode, VPC, code signing, KMS key, and tags.
Cannot add or remove after creation
Cancel
Create function
Info
Tutorials
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Breaking news
Odisha class 10t...
Search
ENG
IN
16:52
02-05-2026

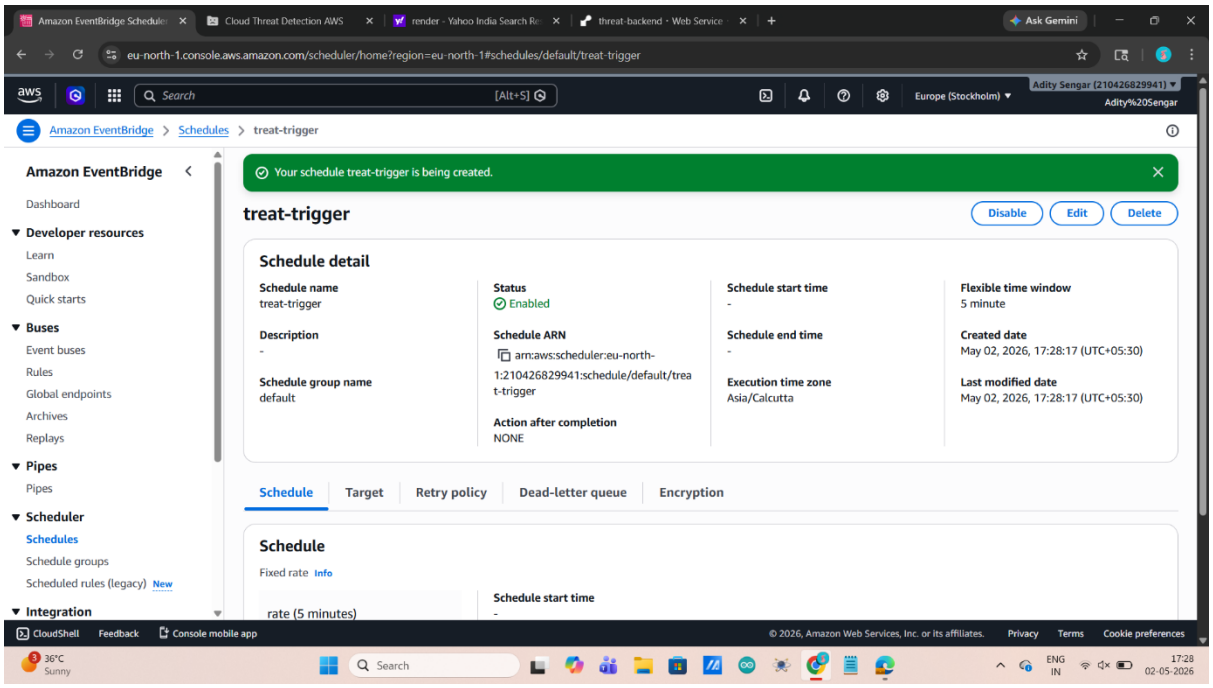

24Online Client
threat-handler | Functions | La
Ask Gemini
eu-north-1.console.aws.amazon.com/lambda/home?region=eu-north-1#/functions/threat-handler?newFunction=true&tab=code
Search
[Alt+S]
Europe (Stockholm)
Adity Sengar (210426829941)
Adity%20Sengar
Lambda > Functions > threat-handler
Successfully created the function "threat-handler". You can now change its code and configuration. To invoke your function with a test event, choose "Test".
threat-handler
Throttle
Copy ARN
Actions
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
threat-handler
Layers
(0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:eu-north-1:210426829941:function:threat-handler
Description
-
Last modified
1 second ago
Code
Test
Monitor
Configuration
Aliases
Versions
Code source Info
Open in Visual Studio Code
Upload from
threat-handler
EXPLORER
THREAT-HANDLER
Info
Tutorials
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
2 cm of rain
Monday
Search
ENG
IN
16:53
02-05-2026


Successfully created the function "threat-handler". You can now change its code and configuration. To invoke your function with a test event, choose "Test".
lambda_function.py
import json
import urllib3
http = urllib3.PoolManager()
def lambda_handler(event, context):
data = {
"type": "failed-login",
"user": "aws-user",
"sourceIP": "unknown",
"time": "auto"
}
url = "https://threat-backend-hulu.onrender.com"
response = http.request(
'POST',
url,
body=json.dumps(data),
headers={'Content-Type': 'application/json'}
)
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
Create a simple web app


The test event "testEvent" was successfully saved.
Code
Test
Monitor
Configuration
Aliases
Versions
Code source
Open in Visual Studio Code
Upload from
Create new test event
Invoke
Save
Event Name
testEvent
Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.
Invocation type
Synchronous
Asynchronous
Deploy (Ctrl+Shift+U)

5. Automation using EventBridge Scheduler

- Created a scheduled rule using: rate(5 minutes)
- Configured target as Lambda function
- Enabled automatic triggering

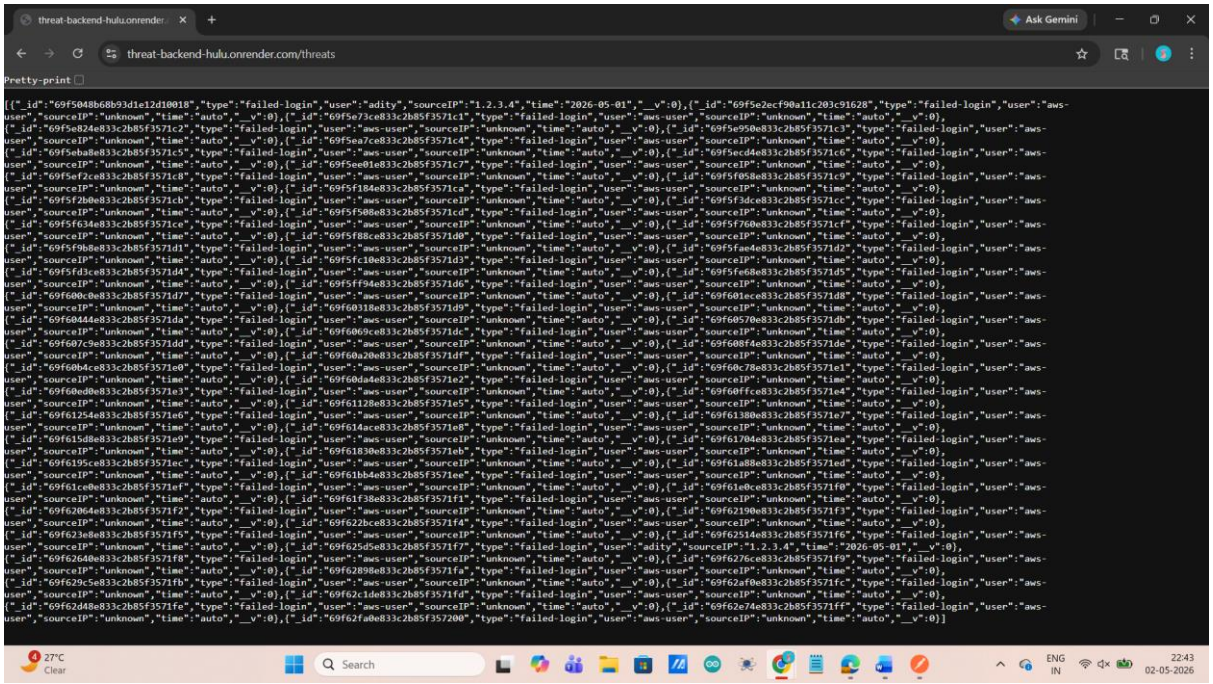


Workflow

1. EventBridge triggers Lambda every 5 minutes
2. Lambda generates threat data
3. Data is sent to backend API
4. Backend stores it in MongoDB
5. Data can be retrieved via /threats endpoint

Output

- Automatic entries generated in database
- API shows increasing threat logs over time



Result

Successfully built an automated cloud-based threat logging system that continuously collects and stores simulated threat data using AWS services.

Future Improvements

- Add real-time dashboard (React)
- Integrate alert system (email/SMS)

- Detect real threats instead of simulated data
- Add authentication and security layers

Conclusion

This project demonstrates the use of serverless architecture and cloud automation to build a scalable and efficient threat detection pipeline.